```python
from decimal import Decimal, ROUND_HALF_UP

CENT = Decimal("0.01")
BONUS_THRESHOLD_LITRES = Decimal("500")
BONUS_PER_LITRE = Decimal("0.50")

def money(value: Decimal) -> Decimal:
    """Round a monetary amount to two decimal places using half-up rounding."""
    return value.quantize(CENT, rounding=ROUND_HALF_UP)

def get_rate_card(month: int) -> dict:
    """Return the rate card for a month (1-12)."""
    if not isinstance(month, int) or not 1 <= month <= 12:
        raise ValueError("month must be an integer from 1 to 12")
    if month in (4, 5, 6):
        return {
            "base_rate": Decimal("32.00"),
            "fat_step_rate": Decimal("0.45"),
            "snf_step_rate": Decimal("0.30"),
        }
    return {
        "base_rate": Decimal("30.00"),
```

```
        "fat_step_rate": Decimal("0.40"),
        "snf_step_rate": Decimal("0.30"),
    }

def _steps(value: Decimal, standard: Decimal) -> int:
    """Number of 0.1 percentage-point steps, rounded to nearest integer."""
    # Decimal's integral rounding is explicit and avoids binary float arithmetic.
    return int(((value - standard) / Decimal("0.1")).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP
    ))

def validate_entry(litres, fat, snf) -> tuple[Decimal, Decimal, Decimal]:
    """Convert values to Decimal and validate non-negative litres and plausible readings."""
    try:
        litres = Decimal(str(litres))
        fat = Decimal(str(fat))
        snf = Decimal(str(snf))
    except Exception as exc:
        raise ValueError("litres, fat and SNF must be numeric") from exc

```

```
    if not all(x.is_finite() for x in (litres, fat, snf)):
        raise ValueError("values must be finite numbers")
    if litres <= 0:
        raise ValueError("litres must be greater than zero")
    # Assumptions: broad plausible ranges; adjust to the lecturer's exact limits if given.
    if not Decimal("0") <= fat <= Decimal("15"):
        raise ValueError("fat must be between 0 and 15 percent")
    if not Decimal("0") <= snf <= Decimal("20"):
        raise ValueError("SNF must be between 0 and 20 percent")
    return litres, fat, snf

def calculate_rate(fat, snf, month: int = 1) -> Decimal:
    """Calculate price per litre from fat/SNF and month-specific rate card."""
    try:
        fat = Decimal(str(fat))
        snf = Decimal(str(snf))
    except Exception as exc:
        raise ValueError("fat and SNF must be numeric") from exc
    if not fat.is_finite() or not snf.is_finite():
        raise ValueError("fat and SNF must be finite numbers")
    card = get_rate_card(month)
```

```
    fat_steps = _steps(fat, Decimal("3.5"))
    snf_steps = _steps(snf, Decimal("8.5"))
    rate = (card["base_rate"]
            + Decimal(fat_steps) * card["fat_step_rate"]
            + Decimal(snf_steps) * card["snf_step_rate"])
    return money(rate)

def calculate_payment(litres, fat, snf, month: int = 1) -> Decimal:
    """Calculate one entry's payable amount, rounded to cents."""
    litres, fat, snf = validate_entry(litres, fat, snf)
    return money(litres * calculate_rate(fat, snf, month))

def calculate_ten_day_total(entries, month: int = 1) -> dict:
    """Calculate itemised payments and a ten-day bonus on all litres if total > 500."""
    entries = list(entries)
    payments = [calculate_payment(e["litres"], e["fat"], e["snf"], month) for e in entries]
    litres_total = sum((Decimal(str(e["litres"])) for e in entries), Decimal("0"))
    milk_total = money(sum(payments, Decimal("0")))
    bonus = money(litres_total * BONUS_PER_LITRE) if litres_total >
```

```
    BONUS_THRESHOLD_LITRES else Decimal("0.00")
    return {
        "litres_total": litres_total,
        "milk_total": milk_total,
        "bonus": bonus,
        "total_payable": money(milk_total + bonus),
        "payments": payments,
    }

if __name__ == "__main__":
    # Small demonstration; normal user interaction is in main.py.
    print("Example payment:", calculate_payment("10", "3.5", "8.5", month=1))
    print("April example:", calculate_payment("10", "3.5", "8.5", month=4))

```

Output

```
Example payment: 300.00
April example: 320.00
```